ANIMEX

Complete Backend Implementation Plan

Node.js / Express backend for Vision, chat, quiz, habitat, metadata, security, and demo readiness

Prepared for backend engineers | April 2026

1. Backend Responsibility in Animex

The Animex backend is the trusted server-side boundary between the Ionic/Vue mobile app and third-party AI services. It does not render the user interface, access the device camera directly, or store personal user profiles. Its responsibility is to receive validated requests from the client, protect API keys, call Google Cloud Vision and Gemini, deterministically map noisy external AI outputs to supported internal animal records, and return concise, app-ready data structures that support the three Animex experiences: visual animal identification, conversational learning, and habitat immersion.

The backend must make the demo reliable in a zoo-like context where camera frames can be blurry, multiple animals may appear in one frame, background noise may interfere with voice input, and network or AI services may fail. The backend therefore owns validation, rate limiting, timeouts, fallback responses, animal metadata, habitat metadata, quiz state, and consistent error formatting.

Area	Backend responsibility	Out of scope for backend
Vision identification	Validate image payloads; call Vision; extract labels, localized objects, web entities; map to internal animal IDs; return confidence, candidates, and fallback messages.	Live camera rendering, client-side frame capture, AR drawing.
Animal metadata	Load curated JSON animal catalog into memory; serve supported animals; provide chat, quiz, and habitat metadata.	Authoring final educational content in the app UI.
Chat and quiz	Build bounded prompts with animal context; call Gemini; validate quiz answers; produce child-friendly fallback responses.	Speech-to-text and text-to-speech execution, which stays on device/WebView.
Habitat	Represent habitats and animal-to-habitat relationships; return asset manifests and missing-data fallbacks.	Real-time 3D rendering or full photorealistic scene generation.
Security/reliability	Keep secrets server-side; CORS; rate limits; request size limits; logging; validation; graceful degradation.	User authentication accounts, payments, persistent PII storage.

2. Backend Architecture and Structure

2.1 Recommended Technology Stack

Layer	Choice	Notes
Runtime	Node.js 20 LTS	Use built-in fetch and AbortController for external timeouts.
Framework	Express 5 + TypeScript	Small, familiar, adequate for local demo and optional deployment.
Validation	zod	Runtime request and config validation with typed inference.
External APIs	Google Cloud Vision REST, Gemini API	Accessed only from backend.
Testing	Vitest + Supertest + Nock/MSW	Fast endpoint and service tests with mocked external APIs.
Logging	pino + pino-http	Structured logs with request IDs and no image/base64 payload logging.
Rate limiting	express-rate-limit	Per-route policies for Vision/chat/quiz.
Security middleware	helmet, cors, express.json limits	Local/demo safe defaults.

2.2 Repository Layout

```
animex-backend/
  src/
    app.ts                      # Express app wiring
    server.ts                   # Process startup and shutdown
    config/
      env.ts                    # zod-validated environment variables
      cors.ts                   # CORS allowlist
      rateLimits.ts             # Per-route limit definitions
    routes/
```

```
      health.routes.ts
      animals.routes.ts
      vision.routes.ts
      chat.routes.ts
      quiz.routes.ts
      habitats.routes.ts
      assets.routes.ts
      admin.routes.ts              # development-only reload endpoint
    controllers/
      animals.controller.ts
      vision.controller.ts
      chat.controller.ts
      quiz.controller.ts
      habitats.controller.ts
    services/
      animalCatalog.service.ts     # load/cache curated animal JSON
      animalMatcher.service.ts     # deterministic Vision-to-animal mapping
      vision.service.ts            # Google Cloud Vision client wrapper
      gemini.service.ts            # Gemini client wrapper
      chat.service.ts              # prompt construction + safety/fallbacks
      quiz.service.ts              # quiz retrieval/generation + validation
      habitat.service.ts           # habitat lookup and asset references
      cache.service.ts             # optional demo TTL cache
    schemas/
      animal.schema.ts
      api.schema.ts
      vision.schema.ts
      chat.schema.ts
      quiz.schema.ts
      habitat.schema.ts
    middleware/
      requestId.ts
      validate.ts
      errorHandler.ts
      rateLimiter.ts
      securityHeaders.ts
    data/
      animals/*.json
      habitats/*.json
      assets/asset-manifest.json
    tests/
      unit/
      integration/
      fixtures/
  .env.example
  package.json
  tsconfig.json
  vitest.config.ts
  README.md
```

2.3 Service Boundaries and Interaction Flow

Component	Owns	Calls	Returns
Routes/controllers	HTTP parsing, headers, status codes, validation middleware attachment.	Service layer only.	Typed API envelopes.
AnimalCatalogService	Loading and caching JSON catalog; lookup by ID; habitat relations; supported species list.	Filesystem at startup or admin reload.	Animal records and indexes.
AnimalMatcherService	Normalization, synonym matching, confidence scoring, ambiguity resolution.	AnimalCatalogService.	MATCHED, AMBIGUOUS, UNSUPPORTED, or LOW_CONFIDENCE results.
VisionService	Google Vision request construction, timeout, parsing raw response fields.	Google Cloud Vision API.	Normalized Vision signals: labels, localized objects, web entities.
GeminiService	Gemini timeout, prompt call, retry policy limited to safe idempotent failures.	Gemini API.	Text or structured JSON according to route.
ChatService	Prompt rules, animal context injection, child-friendly safety	GeminiService + AnimalCatalogService.	Chat answer envelope.

	constraints, fallback text.		
QuizService	Quiz retrieval or generation, answer validation, fallback validation.	AnimalCatalogService, optionally GeminiService.	Quiz question and validation envelope.
HabitatService	Habitat data lookup, asset manifest assembly, missing data fallback.	AnimalCatalogService + local habitat JSON.	Habitat view model.

Primary Vision flow: client captures a throttled JPEG frame -> POST /api/vision/identify -> validation/rate limit -> VisionService calls Google Cloud Vision with a timeout -> AnimalMatcherService maps labels and web entities to internal animals -> response contains a selected animal, candidates, unsupported/ambiguous state, and a UI message.

Primary chat flow: client sends animalId, user message, and bounded recent history -> POST /api/chat -> ChatService injects only curated animal context and concise behavior rules -> GeminiService calls Gemini with a timeout -> response returns text, optional suggested quiz trigger, and fallback metadata.

3. API Design Principles

- All endpoints are versioned under /api/v1 except /healthz and /readyz.
- Every success response uses { "ok": true, "data": ..., "meta": ... }.
- Every failure response uses the common error format defined in Section 11.
- Client must send X-Animex-Client: mobile and X-Request-Id when available. The backend generates X-Request-Id if absent.
- No endpoint accepts raw API keys from the client.
- Base64 image payloads are accepted only as JPEG or PNG data URLs or plain base64 with explicit mimeType.
- The backend returns deterministic animal IDs; the frontend never treats Vision labels as species IDs.

4. Common Headers, Success Envelope, and Error Envelope

Header	Required	Description
Content-Type: application/json	Required for POST	All request bodies are JSON.
X-Animex-Client: mobile	Recommended	Used for CORS/rate-limit debugging and demo telemetry.
X-Request-Id: <uuid>	Optional	Preserved in logs and responses; generated when missing.
Authorization	Not required for demo	Optional future token hook. Do not add before needed.

```
// Success envelope
{
  "ok": true,
  "data": { },
  "meta": {
    "requestId": "req_01HV...",
    "timestamp": "2026-04-29T12:00:00.000Z"
  }
}

// Error envelope
{
  "ok": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "imageBase64 is required and must be a JPEG or PNG payload.",
    "details": [
      { "path": "imageBase64", "reason": "Required" }
    ],
    "retryable": false
  },
```

```
  "meta": {
    "requestId": "req_01HV...",
    "timestamp": "2026-04-29T12:00:00.000Z"
  }
}
```

5. Complete Endpoint Specification

GET /healthz

Field	Specification
Purpose	Process liveness check for local scripts, mobile diagnostics, and deployment health checks.
Request parameters	None.
Request body	None.
Headers	None beyond standard headers.
Example request	GET /healthz
Successful response format	200 text/plain or JSON depending on implementation.
Example successful response	{ "ok": true, "status": "live" }
Failure cases	500 INTERNAL_ERROR if process is unhealthy.
Auth / rate limit	No auth. No rate limit required beyond global abuse guard.
Error response format	Common error envelope from Section 4.

GET /readyz

Field	Specification
Purpose	Readiness check confirming animal catalog is loaded and required API keys are present.
Request parameters	None.
Request body	None.
Headers	None.
Example request	GET /readyz
Successful response format	JSON success envelope with readiness details.
Example successful response	{ "ok": true, "data": { "ready": true, "animalsLoaded": 12, "visionConfigured": true, "geminiConfigured": true } }
Failure cases	503 SERVICE_UNAVAILABLE when catalog missing or API keys absent.
Auth / rate limit	No auth. No rate limit required beyond global abuse guard.
Error response format	Common error envelope from Section 4.

GET /api/v1/version

Field	Specification
Purpose	Returns backend version, build time, and enabled feature flags for the app diagnostics screen.
Request parameters	None.
Request body	None.
Headers	None.
Example request	GET /api/v1/version
Successful response format	JSON success envelope.
Example successful response	{ "ok": true, "data": { "name": "animex-backend", "version": "1.0.0", "environment": "development", "features": { "vision": true, "chat": true, "quiz": true, "habitats": true } } }
Failure cases	500 INTERNAL_ERROR only.
Auth / rate limit	No auth. 60/min per IP.
Error response format	Common error envelope from Section 4.

GET /api/v1/animals

Field	Specification
Purpose	Lists supported animals. Used for debug screens, manual selection fallback, and offline demo verification.
Request parameters	Query: habitatId?: string; q?: string; includeDetails?: boolean=false.
Request body	None.

Headers	None.
Example request	GET /api/v1/animals?habitatId=savannah&includeDetails=false
Successful response format	JSON success envelope with animals array.
Example successful response	{ "ok": true, "data": { "animals": [{ "id": "lion", "displayName": "Lion", "scientificName": "Panthera leo", "habitatIds": ["savannah"], "diet": "carnivore" }] } }
Failure cases	400 VALIDATION_ERROR for invalid query; 500 CATALOG_ERROR.
Auth / rate limit	No auth. 120/min per IP.
Error response format	Common error envelope from Section 4.

GET /api/v1/animals/:animalId

Field	Specification
Purpose	Returns one animal with full metadata for identification, chat context, habitat, and quiz bootstrapping.
Request parameters	Path: animalId string matching catalog ID. Query: includeQuiz?: boolean=true; includeChatContext?: boolean=true.
Request body	None.
Headers	None.
Example request	GET /api/v1/animals/lion
Successful response format	JSON success envelope with animal object.
Example successful response	{ "ok": true, "data": { "animal": { "id": "lion", "displayName": "Lion", "scientificName": "Panthera leo", "facts": ["Lions live in groups called prides."], "habitat": { "primaryHabitatId": "savannah" } } } }
Failure cases	404 ANIMAL_NOT_FOUND; 400 VALIDATION_ERROR; 500 CATALOG_ERROR.
Auth / rate limit	No auth. 120/min per IP.
Error response format	Common error envelope from Section 4.

POST /api/v1/vision/identify

Field	Specification
Purpose	Identifies an animal from a single camera frame and maps Vision results to internal animal IDs.
Request parameters	Query: demoCache?: boolean=false.
Request body	Body: imageBase64 string; mimeType image/jpeg|image/png; clientFrameId string; sessionId string; previousAnimalId? string; crop? {x,y,width,height}; deviceHints? { platform, cameraFacing, width, height }.
Headers	Content-Type JSON. X-Request-Id recommended.
Example request	POST /api/v1/vision/identify with imageBase64, mimeType, clientFrameId, sessionId.
Successful response format	JSON success envelope with status, selectedAnimal, candidates, visionSummary, and uiMessage.
Example successful response	{ "ok": true, "data": { "status": "MATCHED", "selectedAnimal": { "id": "lion", "displayName": "Lion", "confidence": 0.91 }, "candidates": [{ "animalId": "lion", "score": 0.91, "matchedSignals": ["label:lion", "webEntity:Panthera leo"] }], "uiMessage": "This looks like a lion." } }
Failure cases	400 VALIDATION_ERROR; 413 PAYLOAD_TOO_LARGE; 415 UNSUPPORTED_MEDIA_TYPE; 422 MALFORMED_IMAGE; 429 RATE_LIMITED; 504 VISION_TIMEOUT; 503 VISION_UNAVAILABLE; 200 with status UNSUPPORTED/AMBIGUOUS/LOW_CONFIDENCE for valid but inconclusive images.
Auth / rate limit	No user auth. Strict rate limit: 30/min/IP and 12/min/session. Client must throttle to max 1 frame every 2 seconds and must not send a new frame while one is in flight.
Error response format	Common error envelope from Section 4.

POST /api/v1/chat

Field	Specification
Purpose	Answers a child-friendly question about the currently selected animal using Gemini and curated animal context.
Request parameters	None.
Request body	Body: sessionId string; animalId string; message string 1-500 chars; history array max 8 turns; inputMode text|voice; locale?; ageBand? child|teen|general.
Headers	Content-Type JSON. X-Request-Id recommended.

Example request	POST /api/v1/chat { "sessionId":"demo-1", "animalId":"lion", "message":"Where do lions live?", "history":[], "inputMode":"text" }
Successful response format	JSON success envelope with answer, source, suggestedActions, and optional quizPrompt.
Example successful response	{ "ok": true, "data": { "answer": "Lions mostly live in African grasslands and savannahs. They like open places where they can hunt and rest together in prides.", "source": "gemini", "animalId": "lion", "suggestedActions": ["showHabitat", "askQuiz"], "safety": { "filtered": false } } }
Failure cases	400 VALIDATION_ERROR; 404 ANIMAL_NOT_FOUND; 429 RATE_LIMITED; 504 GEMINI_TIMEOUT; 503 GEMINI_UNAVAILABLE returns deterministic fallback answer where possible.
Auth / rate limit	No user auth. 20/min/IP and 12/min/session.
Error response format	Common error envelope from Section 4.

POST /api/v1/quiz/next

Field	Specification
Purpose	Returns the next quiz question for an animal. Prefer curated quiz metadata; optionally generate with Gemini only when catalog lacks enough questions.
Request parameters	None.
Request body	Body: sessionId string; animalId string; difficulty? easy|medium; previousQuestionIds? string[]; mode? animal|habitat.
Headers	Content-Type JSON.
Example request	POST /api/v1/quiz/next { "sessionId":"demo-1", "animalId":"lion", "difficulty":"easy", "mode":"animal" }
Successful response format	JSON success envelope with quiz question.
Example successful response	{ "ok": true, "data": { "question": { "id":"lion-q1", "type":"multiple_choice", "prompt":"What do lions usually eat?", "choices":["Grass","Meat","Fruit"], "answerMode":"text_or_voice" }, "source":"curated" } }
Failure cases	400 VALIDATION_ERROR; 404 ANIMAL_NOT_FOUND; 429 RATE_LIMITED; 503 GEMINI_UNAVAILABLE with curated fallback if available; 404 QUIZ_NOT_FOUND if none.
Auth / rate limit	No user auth. 30/min/IP and 20/min/session.
Error response format	Common error envelope from Section 4.

POST /api/v1/quiz/validate

Field	Specification
Purpose	Validates a user answer against a quiz question. Uses deterministic matching for curated questions and Gemini only for open-ended semantic answers when needed.
Request parameters	None.
Request body	Body: sessionId string; animalId string; questionId string; answer string; inputMode text|voice.
Headers	Content-Type JSON.
Example request	POST /api/v1/quiz/validate { "animalId":"lion", "questionId":"lion-q1", "answer":"meat", "inputMode":"voice" }
Successful response format	JSON success envelope with correctness, feedback, and retry hint.
Example successful response	{ "ok": true, "data": { "correct": true, "score": 1, "feedback": "Correct! Lions are carnivores, which means they eat meat.", "nextAction": "continue" } }
Failure cases	400 VALIDATION_ERROR; 404 ANIMAL_NOT_FOUND; 404 QUIZ_NOT_FOUND; 429 RATE_LIMITED; 504 GEMINI_TIMEOUT for semantic open answer, with fallback to conservative incorrect/needs-review message.
Auth / rate limit	No user auth. 40/min/IP and 25/min/session.
Error response format	Common error envelope from Section 4.

GET /api/v1/habitats

Field	Specification
Purpose	Lists supported habitats and summary data for the AR habitat selection layer.
Request parameters	Query: includeAssets?: boolean=false.
Request body	None.
Headers	None.

Example request	GET /api/v1/habitats?includeAssets=true
Successful response format	JSON success envelope with habitats array.
Example successful response	{ "ok": true, "data": { "habitats": [{ "id":"savannah", "displayName":"Savannah", "summary":"Open grassland with scattered trees.", "assetCount": 5 }] } }
Failure cases	400 VALIDATION_ERROR; 500 HABITAT_DATA_ERROR.
Auth / rate limit	No auth. 120/min/IP.
Error response format	Common error envelope from Section 4.

GET /api/v1/habitats/:habitatId

Field	Specification
Purpose	Returns habitat details, AR asset references, educational text, and fallback information.
Request parameters	Path: habitatId. Query: animalId?: string; includeObjects?: boolean=true.
Request body	None.
Headers	None.
Example request	GET /api/v1/habitats/savannah?animalId=lion
Successful response format	JSON success envelope with habitat object and asset manifest subset.
Example successful response	{ "ok": true, "data": { "habitat": { "id":"savannah", "displayName":"Savannah", "description":"A sunny grassland with scattered trees.", "assets": { "background":"assets/habitats/savannah/bg.webp", "objects": [{"id":"acacia_tree","label":"Acacia tree"}] }, "relatedAnimalIds": ["lion","giraffe"] } } }
Failure cases	404 HABITAT_NOT_FOUND; 400 VALIDATION_ERROR; 500 HABITAT_DATA_ERROR.
Auth / rate limit	No auth. 120/min/IP.
Error response format	Common error envelope from Section 4.

GET /api/v1/animals/:animalId/habitat

Field	Specification
Purpose	Convenience endpoint used by the AR view after identification to fetch the primary habitat for an animal.
Request parameters	Path: animalId. Query: includeAssets?: boolean=true.
Request body	None.
Headers	None.
Example request	GET /api/v1/animals/lion/habitat
Successful response format	JSON success envelope with animal and habitat view model.
Example successful response	{ "ok": true, "data": { "animalId":"lion", "habitat": { "id":"savannah", "displayName":"Savannah", "assets": { "background":"assets/habitats/savannah/bg.webp" } }, "fallback": false } }
Failure cases	404 ANIMAL_NOT_FOUND; 404 HABITAT_NOT_FOUND; returns generic habitat fallback with fallback=true if configured.
Auth / rate limit	No auth. 120/min/IP.
Error response format	Common error envelope from Section 4.

GET /api/v1/assets/manifest

Field	Specification
Purpose	Returns static asset metadata so the frontend can prefetch common habitat images and overlays before the demo.
Request parameters	Query: habitatId?: string; animalId?: string.
Request body	None.
Headers	None.
Example request	GET /api/v1/assets/manifest?habitatId=savannah
Successful response format	JSON success envelope with asset entries.
Example successful response	{ "ok": true, "data": { "assets": [{ "id":"savannah-bg", "type":"image", "url":"/assets/habitats/savannah/bg.webp", "bytes": 245000, "preload": true }] } }
Failure cases	400 VALIDATION_ERROR; 500 ASSET_MANIFEST_ERROR.
Auth / rate limit	No auth. 120/min/IP. Static assets may also be served through Express static with cache headers.
Error response format	Common error envelope from Section 4.

POST /api/v1/admin/reload-catalog

Field	Specification
Purpose	Development-only endpoint that reloads JSON animal/habitat data without restarting the server. Must be disabled in production.
Request parameters	None.
Request body	Body: optional { "reloadAnimals": true, "reloadHabitats": true }.
Headers	X-Admin-Token required in non-local environments if enabled.
Example request	POST /api/v1/admin/reload-catalog { "reloadAnimals": true, "reloadHabitats": true }
Successful response format	JSON success envelope with counts.
Example successful response	{ "ok": true, "data": { "animalsLoaded": 12, "habitatsLoaded": 5, "reloadedAt":"2026-04-29T12:00:00.000Z" } }
Failure cases	403 FORBIDDEN if disabled or bad token; 500 CATALOG_ERROR/HABITAT_DATA_ERROR.
Auth / rate limit	Disabled in production. Localhost only. 5/min/IP.
Error response format	Common error envelope from Section 4.

6. Animal Data Model and JSON Schema

Animal metadata is the core deterministic layer of Animex. Vision APIs can return generic labels such as "animal", "big cat", "mammal", web entity names such as "Panthera leo", or noisy annotations. The backend must never expose these strings as canonical species. Instead, every supported species has a stable internal ID and curated matching metadata.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://animex.local/schemas/animal.schema.json",
  "title": "AnimexAnimal",
  "type": "object",
  "required": [
    "id",
    "displayName",
    "scientificName",
    "aliases",
    "vision",
    "habitat",
    "diet",
    "facts",
    "conservationStatus",
    "assets",
    "quiz",
    "chatContext"
  ],
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string",
      "pattern": "^[a-z0-9]+(?:-[a-z0-9]+)*$",
      "description": "Stable internal animal ID used by all backend and frontend routes."
    },
    "displayName": {
      "type": "string",
      "minLength": 1
    },
    "scientificName": {
      "type": "string",
      "minLength": 1
    },
    "taxonomicClass": {
      "type": "string",
      "examples": [
        "Mammalia",
        "Aves",
        "Reptilia"
      ]
    },
    "aliases": {
```

```
    "type": "array",
    "items": {
      "type": "string"
    },
    "minItems": 1,
    "description": "Human-readable synonyms and common names."
  },
  "vision": {
    "type": "object",
    "required": [
      "supportedLabels",
      "supportedWebEntities",
      "negativeLabels",
      "thresholds"
    ],
    "properties": {
      "supportedLabels": {
        "type": "array",
        "items": {
          "type": "object",
          "required": [
            "name",
            "weight"
          ],
          "properties": {
            "name": {
              "type": "string"
            },
            "weight": {
              "type": "number",
              "minimum": 0,
              "maximum": 1
            }
          }
        }
      },
      "supportedWebEntities": {
        "type": "array",
        "items": {
          "type": "object",
          "required": [
            "name",
            "weight"
          ],
          "properties": {
            "name": {
              "type": "string"
            },
            "weight": {
              "type": "number",
              "minimum": 0,
              "maximum": 1
            }
          }
        }
      },
      "negativeLabels": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "Labels that reduce confidence, e.g. tiger for lion."
      },
      "thresholds": {
        "type": "object",
        "required": [
          "minMatchScore",
          "strongMatchScore",
          "ambiguityDelta"
        ],
        "properties": {
```

```
        "minMatchScore": {
          "type": "number",
          "minimum": 0,
          "maximum": 1
        },
        "strongMatchScore": {
          "type": "number",
          "minimum": 0,
          "maximum": 1
        },
        "ambiguityDelta": {
          "type": "number",
          "minimum": 0,
          "maximum": 1
        }
      }
    }
  }
},
"habitat": {
  "type": "object",
  "required": [
    "primaryHabitatId",
    "habitatIds",
    "summary",
    "climate",
    "mapRefs"
  ],
  "properties": {
    "primaryHabitatId": {
      "type": "string"
    },
    "habitatIds": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "summary": {
      "type": "string"
    },
    "climate": {
      "type": "string"
    },
    "mapRefs": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  }
},
"diet": {
  "type": "object",
  "required": [
    "type",
    "description",
    "examples"
  ],
  "properties": {
    "type": {
      "enum": [
        "herbivore",
        "carnivore",
        "omnivore",
        "insectivore",
        "piscivore",
        "unknown"
      ]
    },
    "description": {
```

```
      "type": "string"
    },
    "examples": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  }
},
"facts": {
  "type": "array",
  "items": {
    "type": "string"
  },
  "minItems": 3
},
"conservationStatus": {
  "type": "object",
  "required": [
    "label",
    "source",
    "summary"
  ],
  "properties": {
    "label": {
      "type": "string"
    },
    "source": {
      "type": "string"
    },
    "summary": {
      "type": "string"
    }
  }
},
"assets": {
  "type": "object",
  "properties": {
    "thumbnail": {
      "type": "string"
    },
    "silhouette": {
      "type": "string"
    },
    "habitatBackground": {
      "type": "string"
    },
    "arObjects": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  }
},
"quiz": {
  "type": "object",
  "required": [
    "difficulty",
    "questions"
  ],
  "properties": {
    "difficulty": {
      "enum": [
        "easy",
        "medium"
      ]
    },
    "questions": {
      "type": "array",
```

```
        "items": {
          "type": "object",
          "required": [
            "id",
            "type",
            "prompt",
            "acceptedAnswers",
            "feedback"
          ],
          "properties": {
            "id": {
              "type": "string"
            },
            "type": {
              "enum": [
                "multiple_choice",
                "open_text",
                "yes_no"
              ]
            },
            "prompt": {
              "type": "string"
            },
            "choices": {
              "type": "array",
              "items": {
                "type": "string"
              }
            },
            "acceptedAnswers": {
              "type": "array",
              "items": {
                "type": "string"
              }
            },
            "feedback": {
              "type": "string"
            },
            "habitatRelated": {
              "type": "boolean"
            }
          }
        }
      }
    }
  },
  "chatContext": {
    "type": "object",
    "required": [
      "safeSummary",
      "allowedTopics",
      "blockedTopics",
      "toneRules",
      "maxAnswerSentences"
    ],
    "properties": {
      "safeSummary": {
        "type": "string"
      },
      "allowedTopics": {
        "type": "array",
        "items": {
          "type": "string"
        }
      },
      "blockedTopics": {
        "type": "array",
        "items": {
          "type": "string"
        }
      },
```

```
        "toneRules": {
          "type": "array",
          "items": {
            "type": "string"
          }
        },
        "maxAnswerSentences": {
          "type": "integer",
          "minimum": 1,
          "maximum": 5
        }
      }
    }
  }
}
```

6.1 Example Animal Record

```
{
  "id": "lion",
  "displayName": "Lion",
  "scientificName": "Panthera leo",
  "taxonomicClass": "Mammalia",
  "aliases": [
    "lion",
    "african lion",
    "lioness",
    "male lion",
    "big cat"
  ],
  "vision": {
    "supportedLabels": [
      {
        "name": "lion",
        "weight": 1.0
      },
      {
        "name": "big cat",
        "weight": 0.55
      },
      {
        "name": "terrestrial animal",
        "weight": 0.15
      }
    ],
    "supportedWebEntities": [
      {
        "name": "Panthera leo",
        "weight": 1.0
      },
      {
        "name": "Lion",
        "weight": 0.95
      },
      {
        "name": "African lion",
        "weight": 0.95
      }
    ],
    "negativeLabels": [
      "tiger",
      "cheetah",
      "leopard",
      "domestic cat"
    ],
    "thresholds": {
      "minMatchScore": 0.62,
      "strongMatchScore": 0.82,
      "ambiguityDelta": 0.12
    }
  },
```

```
  "habitat": {
    "primaryHabitatId": "savannah",
    "habitatIds": [
      "savannah",
      "grassland"
    ],
    "summary": "Open grasslands and savannahs with scattered trees.",
    "climate": "Warm, dry to seasonal rainfall",
    "mapRefs": [
      "africa/east",
      "africa/south"
    ]
  },
  "diet": {
    "type": "carnivore",
    "description": "Lions eat meat and hunt prey such as antelope and zebra.",
    "examples": [
      "zebra",
      "antelope",
      "wildebeest"
    ]
  },
  "facts": [
    "Lions live in groups called prides.",
    "Male lions often have manes.",
    "Lions rest for many hours each day."
  ],
  "conservationStatus": {
    "label": "Vulnerable",
    "source": "curated project metadata",
    "summary": "Lion populations face threats from habitat loss and conflict with humans."
  },
  "assets": {
    "thumbnail": "assets/animals/lion/thumb.webp",
    "silhouette": "assets/animals/lion/silhouette.svg",
    "habitatBackground": "assets/habitats/savannah/bg.webp",
    "arObjects": [
      "acacia_tree",
      "tall_grass",
      "rock"
    ]
  },
  "quiz": {
    "difficulty": "easy",
    "questions": [
      {
        "id": "lion-q1",
        "type": "multiple_choice",
        "prompt": "What do lions usually eat?",
        "choices": [
          "Grass",
          "Meat",
          "Leaves"
        ],
        "acceptedAnswers": [
          "meat",
          "animals",
          "prey"
        ],
        "feedback": "Correct. Lions are carnivores."
      }
    ]
  },
  "chatContext": {
    "safeSummary": "A lion is a large carnivorous cat that lives mostly in African savannahs and grasslands.",
    "allowedTopics": [
      "habitat",
      "diet",
      "behavior",
      "appearance",
      "conservation"
```

```
    ],
    "blockedTopics": [
      "graphic hunting detail",
      "dangerous handling advice"
    ],
    "toneRules": [
      "Use simple words",
      "Keep answers short",
      "Be encouraging"
    ],
    "maxAnswerSentences": 3
  }
}
```

7. Animal-Matching Layer Design

7.1 Normalization

- Convert all Vision labels, localized object names, and web entity descriptions to lowercase.
- Trim whitespace, remove punctuation except hyphen inside words, collapse repeated spaces, and remove accents.
- Singularize simple plurals where safe: "lions" -> "lion", "zebras" -> "zebra". Keep scientific names intact.
- Map common aliases through a synonym index built at startup from animal.aliases, vision.supportedLabels.name, and vision.supportedWebEntities.name.
- Ignore generic labels for species selection unless they are paired with a strong species-specific signal. Generic labels include animal, wildlife, mammal, vertebrate, organism, zoo, nature, fauna.

7.2 Signal Extraction from Vision

Vision field	Use	Weight handling
labelAnnotations[].description + score	Useful for common animal labels; often generic.	score * configured label weight.
localizedObjectAnnotations[].name + score	Useful for detecting animal/object presence and multiple subjects.	score * 0.65 unless exact animal alias.
webDetection.webEntities[].description + score	Often strongest for species/scientific names.	score normalized to 0..1, then * configured web entity weight.
webDetection.bestGuessLabels[].label	Useful when web entities are sparse.	0.5 default confidence unless duplicate confirms another signal.
safeSearchAnnotation	Only used to reject inappropriate/malformed content if needed.	No animal score.

7.3 Scoring Algorithm

```
function matchAnimal(visionSignals, catalog): MatchResult {
  const normalizedSignals = normalizeVisionSignals(visionSignals)
  const candidateScores = []

  for (animal of catalog.animals) {
    let score = 0
    let matchedSignals = []

    for (signal of normalizedSignals) {
      if (animal.exactAliasIndex.has(signal.text)) {
        score += signal.confidence * 1.0
        matchedSignals.push(signal)
      }
      if (animal.labelIndex.has(signal.text)) {
        score += signal.confidence * animal.labelIndex.get(signal.text).weight
        matchedSignals.push(signal)
      }
      if (animal.webEntityIndex.has(signal.text)) {
        score += signal.confidence * animal.webEntityIndex.get(signal.text).weight
        matchedSignals.push(signal)
      }
```

```
      if (animal.negativeLabelIndex.has(signal.text)) {
        score -= signal.confidence * 0.6
      }
    }

    score = clamp(score / Math.max(1, matchedSignals.length), 0, 1)
    candidateScores.push({ animalId: animal.id, score, matchedSignals })
  }

  candidateScores.sort(descendingScore)
  const top = candidateScores[0]
  const second = candidateScores[1]
  const thresholds = catalog.get(top.animalId).vision.thresholds

  if (!top || top.score < thresholds.minMatchScore) return LOW_CONFIDENCE
  if (second && top.score - second.score < thresholds.ambiguityDelta) return AMBIGUOUS
  if (top.score >= thresholds.strongMatchScore) return MATCHED
  return MATCHED_LOW_CERTAINTY
}
```

7.4 Unsupported, Ambiguous, and Fallback Handling

Situation	Detection rule	Response behavior
Unsupported animal	Signals are animal-like but no supported animal passes minMatchScore.	Return HTTP 200 with status UNSUPPORTED, selectedAnimal null, top external labels, and message: "I can see an animal, but I do not support that one yet."
Low confidence	Top internal candidate is below minMatchScore.	Return status LOW_CONFIDENCE and prompt user to move closer, improve lighting, or center the animal.
Ambiguous match	Top two candidates are within ambiguityDelta, e.g. leopard vs cheetah.	Return status AMBIGUOUS with candidates and ask frontend to show selection buttons or prompt user to choose.
Multiple subjects	Localized object annotations show multiple animals or client sends selection crop.	Return candidates with bounding boxes when available; frontend asks user to tap/say target.
Previous confirmed animal	previousAnimalId is present and still scores above minMatchScore * 0.85.	Prefer continuity unless a different animal exceeds previous by at least 0.2.
External API failure	Vision timeout/unavailable.	Return 503/504 error envelope or cached result if demoCache enabled and matching image hash exists.

8. Vision Implementation

Requirement	Implementation detail
Input format	POST /api/v1/vision/identify accepts JSON with imageBase64 and mimeType. imageBase64 may be a data URL or raw base64. Supported mime types: image/jpeg and image/png. JPEG preferred.
Maximum payload size	Default 3 MB decoded image size; hard express JSON limit 4 MB. Return 413 PAYLOAD_TOO_LARGE above limit.
Image validation	Decode base64, verify magic bytes, reject SVG/HEIC/GIF/PDF, optionally inspect dimensions. Minimum 160x160, maximum 2048x2048 after client compression.
Vision timeout	2500 ms for demo default; configurable VISION_TIMEOUT_MS. Abort request on timeout.
Frame throttling expectation	Client sends at most one frame every 2 seconds during scanning. Backend enforces per-session limits but client must also prevent duplicate in-flight calls.
Duplicate prevention	Client maintains visionInFlight boolean. Backend also de-duplicates by sessionId + clientFrameId and optional image hash for 10 seconds.
Supported response fields	labelAnnotations, localizedObjectAnnotations, webDetection.webEntities, webDetection.bestGuessLabels, safeSearchAnnotation.
Mapping	VisionService converts raw Google response to VisionSignal[]; AnimalMatcherService maps signals to internal animal IDs.

Optional demo cache	If DEMO_CACHE_ENABLED=true, cache sha256(image bytes prefix/full bytes) -> MatchResult for 5 minutes to stabilize repeated demo frames and reduce costs.
Graceful degradation	On Vision timeout/unavailable, return cached match when available. Otherwise return fallback message instructing the user to retry or select an animal manually.

9. Chat and Quiz Implementation

9.1 Chat Endpoint Behavior

POST /api/v1/chat returns short, child-friendly answers grounded in the selected animal record. The backend should not let the model invent species identity, conservation status, or habitat if the catalog has no data. It should say that the app does not know rather than fabricate.

Topic	Rule
Gemini model use	Use Gemini for natural-language responses only after animalId is validated against the catalog.
Timeout	3000 ms default; configurable GEMINI_TIMEOUT_MS. No more than one retry for transient 429/503 if still inside timeout budget.
History	Accept max 8 recent turns and max 500 chars per user message. Truncate older turns.
Prompt construction	System prompt + animal context + safety constraints + recent history + user message. Do not include raw Vision labels as truth.
Animal context injection	Inject displayName, scientificName, habitat summary, diet, facts, conservation summary, and chatContext safeSummary.
Answer length	Max 3 sentences for child mode; max 5 for general mode.
Safety constraints	No graphic predation details, no instructions to approach/feed/touch animals, no medical/legal advice, no unsafe zoo behavior.
Fallback	If Gemini fails, use deterministic catalog answer by matching keywords: habitat, eat/diet, behavior, facts, conservation. Otherwise return a polite retry/text fallback.

9.2 Prompt Template

```
SYSTEM:
You are Animex, a friendly wildlife guide for children at a zoo.
Answer only about the selected animal unless the user asks to switch animals.
Use the curated animal context as the source of truth.
Never tell the child to touch, feed, scare, or approach animals.
Keep the answer under {maxAnswerSentences} short sentences.
If the answer is not in the context, say you are not sure and suggest asking another question.

ANIMAL CONTEXT:
ID: {animal.id}
Name: {animal.displayName}
Scientific name: {animal.scientificName}
Habitat: {animal.habitat.summary}
Diet: {animal.diet.description}
Facts: {animal.facts.join(" | ")}
Conservation: {animal.conservationStatus.summary}
Allowed topics: {animal.chatContext.allowedTopics}
Blocked topics: {animal.chatContext.blockedTopics}

RECENT HISTORY:
{boundedHistory}

USER QUESTION:
{message}
```

9.3 Quiz Logic

Area	Implementation

Question source	Use curated animal.quiz.questions first. Generate with Gemini only when curated count is insufficient and GEMINI_QUIZ_GENERATION_ENABLED=true. Generated quiz must be validated against schema and never persisted automatically.
Question selection	Avoid previousQuestionIds; prefer habitat-related question when mode=habitat; otherwise rotate deterministic seed by sessionId.
Answer validation	For multiple_choice and yes_no, normalize answer and compare to acceptedAnswers and choices. For open_text, use token overlap/synonyms first; use Gemini semantic validation only if deterministic score is inconclusive.
Feedback	Always provide encouraging feedback. For wrong answers, include the correct fact in one sentence.
Gemini failure	If Gemini validation fails, return correct=false only when deterministic validation says wrong. If uncertain, return correct=null with message asking user to try again or use text.

10. Habitat Implementation

Habitats are curated JSON records referenced by animals through habitat.primaryHabitatId and habitat.habitatIds. The backend returns lightweight view models for AR.js: a background image, optional foreground/label objects, ambient sound references if available, and simple educational text. The frontend performs all rendering.

```
// habitat.schema.json shape
{
  "id": "savannah",
  "displayName": "Savannah",
  "summary": "Open grassland with scattered trees.",
  "description": "A warm habitat where lions can rest, hide in tall grass, and hunt prey.",
  "climate": "Warm with seasonal rain",
  "regions": ["East Africa", "Southern Africa"],
  "assetRefs": {
    "background": "assets/habitats/savannah/bg.webp",
    "ambientSound": "assets/habitats/savannah/ambient.mp3",
    "objects": [
      { "id": "acacia_tree", "label": "Acacia tree", "asset": "assets/habitats/savannah/acacia.webp" },
      { "id": "tall_grass", "label": "Tall grass", "asset": "assets/habitats/savannah/grass.webp" }
    ]
  },
  "fallbackAsset": "assets/habitats/generic/generic-nature.webp",
  "relatedAnimalIds": ["lion", "giraffe", "zebra"]
}
```

Requirement	Design
Animal-to-habitat relationship	Animal records reference one primaryHabitatId and many habitatIds. Habitat records include relatedAnimalIds for reverse lookup. Validate both directions in tests.
Missing habitat data	Return fallback=true with generic habitat summary and generic asset. Log warning with animalId and missing habitatId.
Assets	Use URLs or relative references from asset manifest. Do not inline binary assets in API responses.
AR object addition	When the user asks for an object, chat can return suggestedActions with object IDs from habitat.assetRefs.objects.
Habitat quiz	Quiz mode habitat selects habitatRelated questions first and asks about differences between enclosure and natural habitat.

11. Security and Environment

11.1 Environment Variables

Variable	Required	Example	Purpose
NODE_ENV	Yes	development	Enables dev-only behavior and production restrictions.

PORT	Yes	3000	HTTP port.
CORS_ORIGINS	Yes	http://localhost:8100,capacitor://localhost	Comma-separated allowlist.
GOOGLE_CLOUD_VISION_API_KEY	Yes for Vision	***	Server-side Vision credential.
GEMINI_API_KEY	Yes for chat/quiz generation	***	Server-side Gemini credential.
VISION_TIMEOUT_MS	No	2500	Abort Vision calls.
GEMINI_TIMEOUT_MS	No	3000	Abort Gemini calls.
MAX_IMAGE_BYTES	No	3145728	Decoded max image size.
RATE_LIMIT_WINDOW_MS	No	60000	Rate limit window.
DEMO_CACHE_ENABLED	No	true	Enable short-lived Vision cache.
ADMIN_RELOAD_TOKEN	Production only if admin route enabled	***	Protects reload endpoint.
LOG_LEVEL	No	info	pino log level.

11.2 Required Controls

- Secrets are stored only in .env files or deployment secret managers. .env is excluded from version control. .env.example contains names, not real keys.
- CORS allows only the Ionic dev server, Capacitor origins, and explicit demo origins. Do not use wildcard CORS in production.
- Validate every request with zod before controller logic. Reject unknown properties for image/chat/quiz bodies.
- Never log imageBase64, full prompts, API keys, or raw child-entered personal details. Log requestId, route, status, duration, animalId, result status, and external API latency.
- Use helmet; disable x-powered-by; set JSON body limits; return 413 for payload over limit.
- Enforce media validation by magic bytes, MIME type, decoded size, and dimensions.
- Use rate limits per IP and per sessionId for Vision and Gemini routes.
- Production disables /api/v1/admin/reload-catalog unless explicitly enabled and token-protected.

12. Error Handling and Fallback Behavior

Error code	HTTP	When it occurs	Fallback/response behavior
VALIDATION_ERROR	400	Request shape, query, or path parameter invalid.	Return details array with zod paths.
ANIMAL_NOT_FOUND	404	Unknown animalId.	Prompt frontend to clear selected animal or open manual selection.
HABITAT_NOT_FOUND	404 or 200 fallback	Habitat ID missing.	Use generic habitat fallback for animal habitat route; 404 for direct habitat route unless fallback requested.
UNSUPPORTED_MEDIA_TYPE	415	mimeType not JPEG/PNG.	Ask client to send JPEG.
PAYLOAD_TOO_LARGE	413	Image/body exceeds configured size.	Ask client to compress frame.
MALFORMED_IMAGE	422	Base64 cannot decode or magic bytes invalid.	Ask client to capture again.
RATE_LIMITED	429	Route limit exceeded.	Return retryAfterSeconds.
VISION_TIMEOUT	504	Vision exceeds timeout.	Use recent cache if enabled; otherwise ask user to retry.
VISION_UNAVAILABLE	503	Vision network/API error.	Use cache/manual selection fallback.
GEMINI_TIMEOUT	504	Gemini exceeds timeout.	Use deterministic catalog answer or quiz fallback.
GEMINI_UNAVAILABLE	503	Gemini API/network error.	Use deterministic fallback and mark source=fallback.
CATALOG_ERROR	500	Animal JSON missing/invalid at startup or reload.	readyz fails; app can show maintenance message.
INTERNAL_ERROR	500	Unexpected exception.	Generic message with requestId; log stack server-side only.

12.1 Graceful Degradation Strategy

- Vision fails: show retry guidance and allow manual animal selection from /api/v1/animals. If demo cache has a recent match, return cached result with meta.cached=true.
- Gemini fails: answer using curated animal fields. Example: for "what does it eat", return animal.diet.description. For unknown questions, ask the child to try a simpler question or use text input.
- Habitat missing: return generic nature background and educational text from animal.habitat.summary. Do not block the whole AR experience.
- Quiz generation fails: use curated questions. If none exist, disable quiz prompt for that animal and continue chat/habitat functionality.
- Catalog fails at startup: /readyz returns 503 and server logs clear schema errors before demo.

13. Performance and Reliability Requirements

Requirement	Target
Animal catalog cache	Load and validate all animal/habitat JSON at startup. Keep in memory with indexes by id, alias, label, web entity, and habitat.
Vision latency budget	Aim p95 backend processing under 3500 ms including external Vision. Timeout at 2500 ms external call plus local processing.
Gemini latency budget	Aim p95 under 4000 ms. Timeout external call at 3000 ms.
Frame throttling	Frontend max 1 frame/2s; backend session limit 12/min; reject bursts with 429.
Duplicate Vision calls	Frontend in-flight guard; backend caches clientFrameId for 10 seconds and optional image hash for 5 minutes.
Max image payload	3 MB decoded. Recommend frontend JPEG quality 0.7 and max dimension 1280 px.
Logging	Structured pino logs: requestId, route, status, durationMs, externalDurationMs, sessionId hash, animalId, matchStatus.
Monitoring	For deployment: expose /readyz, count Vision/Gemini failures, rate-limit hits, fallback usage, and average latency. For local demo: console logs plus optional JSON log file.
Startup validation	Server must fail fast if catalog JSON schema invalid or required env missing for enabled features.
No persistent PII	Do not persist images, audio, chat history, or child personal data.

14. Testing Strategy

Test category	Coverage	Acceptance
Unit tests	AnimalMatcherService normalization, scoring, ambiguity, unsupported detection, previousAnimal continuity.	All edge cases deterministic; branch coverage for matcher >90%.
Schema tests	Animal and habitat JSON schema validation; invalid fixtures fail with helpful errors.	Catalog load fails fast on malformed records.
Integration tests	VisionService and GeminiService with mocked HTTP; timeout and error mapping.	No real external API calls in CI.
Endpoint tests	Supertest for all routes: success, validation, not found, rate limit, error envelope.	Every endpoint has at least one success and two failure tests.
Mock Vision API tests	Labels only, web entities only, generic labels, conflicting labels, empty result, timeout, 503.	Correct match status for each fixture.
Mock Gemini tests	Chat success, safety refusal, timeout fallback, unavailable fallback, schema-invalid generated quiz.	Fallbacks are child-friendly and do not crash.
Quiz tests	Curated next question, previous question avoidance, multiple-choice validation, voice typo normalization, semantic fallback.	Correct feedback and score behavior.
Habitat tests	Animal-to-habitat lookup, missing habitat fallback, asset manifest filtering.	AR view can always receive a usable payload for supported animals.
Security/validation tests	Payload too large, unsupported MIME, malformed base64, unknown fields, CORS	Dangerous/invalid requests rejected without leaking secrets.

	allow/deny, admin disabled.	
Reliability tests	Duplicate clientFrameId, in-flight-like repeated calls, cache hit, rate-limit hit.	Stable behavior during demo bursts.

15. Delivery Milestones

Phase	Deliverables	Acceptance criteria
1. Backend skeleton and environment setup	TypeScript Express app; env validation; health/ready/version routes; pino logging; CORS; base error handler; README with run commands.	npm run dev starts; /healthz live; /readyz reports missing config clearly; tests run in CI/local.
2. Animal data model and metadata loading	Animal JSON schema; habitat schema; sample records for demo animals; AnimalCatalogService; startup validation; /animals endpoints.	Invalid JSON blocks startup; supported animals list returns stable IDs; at least 5 common zoo animals have complete metadata.
3. Vision integration and animal matching	VisionService; /vision/identify; matcher indexes; normalization; confidence thresholds; ambiguous/unsupported responses; demo cache; Vision mocks.	Lion/zebra/giraffe/tiger test fixtures map correctly; unsupported fixture returns UNSUPPORTED; no duplicate frame processing by clientFrameId.
4. Chat and quiz implementation	GeminiService; /chat; prompt template; deterministic fallback; /quiz/next; /quiz/validate; curated quiz records.	Chat answers are short and grounded; Gemini timeout returns fallback; quiz can run full question/answer loop without Gemini.
5. Habitat implementation	Habitat records; /habitats; /animals/:id/habitat; /assets/manifest; generic fallback asset.	Each demo animal returns a habitat payload with asset refs; missing habitat does not crash AR flow.
6. Security, validation, and error handling	Route-specific zod validation; media validation; request size limits; rate limits; admin route restrictions; unified error envelope.	Malformed/oversized images rejected; CORS is not wildcard; all errors include requestId and no secrets.
7. Testing and stabilization	Unit/integration/endpoint/security tests; mocked external APIs; test fixtures; latency and fallback checks.	All tests pass; coverage thresholds met for matcher and services; no real API calls in CI.
8. Demo readiness and deployment	.env.example; demo seed data; optional Dockerfile; local run script; smoke-test checklist; frontend integration notes.	Team can start backend, run app, identify supported animals, ask chat questions, answer quiz, and load habitat assets without code changes.

16. Definition of Done

The Animex backend is complete only when all of the following are true:

- The server runs locally with documented commands and exposes /healthz, /readyz, and /api/v1/version.
- All required environment variables are documented in .env.example and validated at startup.
- All supported animal and habitat JSON files validate against schemas and are loaded into memory at startup.
- The backend exposes every endpoint specified in this plan with the documented request/response shape and common error envelope.
- Vision identification accepts validated JPEG/PNG frames, enforces size and rate limits, calls Vision through the backend only, times out cleanly, and maps noisy Vision outputs to deterministic internal animal IDs.
- The animal matching layer handles normalized labels, aliases, web entities, confidence thresholds, unsupported animals, ambiguous matches, and fallback UI messages.
- Chat uses curated animal context, short child-friendly prompt rules, Gemini timeout limits, safety constraints, and deterministic fallback behavior.
- Quiz endpoints can return and validate curated quiz questions without Gemini and can gracefully handle Gemini failures when semantic validation or generation is enabled.
- Habitat endpoints return usable habitat data and asset references for every demo animal, with a generic fallback for missing data.
- Secrets never leave the backend, CORS is allowlisted, payload size is limited, request validation is enforced, and no images/audio/chat histories are persisted.

- Rate limiting, duplicate Vision request prevention, external API timeouts, and optional demo caching are implemented and tested.
- Unit, integration, endpoint, mock Vision, mock Gemini, fallback, security, and validation tests pass locally and in CI.
- A smoke-test checklist proves the full flow: identify animal -> receive description -> ask chat question -> answer quiz -> retrieve habitat assets.
- Backend logs provide enough requestId-based detail to debug failures without logging secrets, images, or sensitive user content.